

Aguará: robot móvil autónomo de aproximación y evasión basado en detección de objetos con una red neuronal ejecutada a bordo

Proyecto Final – Cátedra de Inteligencia Artificial aplicada a la Robótica

Federico Morán¹ y Juan Barboza¹

¹Maestría en Ciencias de la Inteligencia Artificial, Facultad de Ingeniería, Universidad Nacional de Asunción, San Lorenzo, Paraguay .

Contributing authors: federico.moran@uc.edu.py;
juan.barboza@uc.edu.py;

Resumen

Se presenta el diseño, la construcción y la evaluación de Aguará, un robot móvil de tracción diferencial que busca un objeto objetivo, se aproxima a él y evade otro, considerado amenaza, decidiendo todo a bordo. La percepción usa la red de detección YOLOv8n, preentrenada en COCO y ejecutada en la CPU de una Raspberry Pi 5; una máquina de cuatro estados arbitra el comportamiento, con prioridad incondicional de la evasión, y una ley de control proporcional sobre la imagen genera las consignas de avance y giro. Un microcontrolador ESP32 las traduce a potencia para cada rueda, mide la distancia frontal con un sensor ultrasónico y detiene los motores si deja de recibir órdenes. El sistema se evaluó con 79 verificaciones automáticas sin hardware y con 24 corridas registradas en el piso. El lazo completo opera a unas 10 imágenes por segundo, con unos 150 ms de latencia. El robot confirma un objeto 0,2 s después de verlo, alcanza el objetivo desde 1 m en 2 a 3 s y se detiene a unos 30 cm; ante la amenaza inicia la maniobra de evasión en 0,2 s y, con el sensor ultrasónico, interrumpe el avance ante obstáculos. Se discuten sus limitaciones.

Palabras clave: robótica móvil, detección de objetos, YOLOv8, sistemas embebidos, robótica basada en comportamientos, control visual

1 Introducción

La detección de objetos con redes convolucionales de una sola etapa permite hoy que un robot pequeño reconozca lo que tiene delante con una cámara común [1, 2]. El paso que sigue, y que interesa a la robótica móvil, es cerrar el lazo: que esa detección se convierta en una decisión y en un movimiento, en tiempo real y sin una computadora externa. Ese paso obliga a resolver problemas que no aparecen al evaluar la red sobre un conjunto de imágenes: el retardo entre la imagen y la acción, la pérdida momentánea de las detecciones, los actuadores que no responden de forma ideal y la alimentación de un cómputo exigente con baterías.

Este trabajo aborda ese problema con un robot diferencial de bajo costo que exhibe dos conductas opuestas ante dos objetos: se aproxima a uno y huye del otro. Se lo denominó *Aguará*, “zorro” en guaraní, por el animal que se acerca a lo que busca y escapa de lo que lo amenaza. El caso es deliberadamente simple en lo perceptivo — dos clases de un modelo preentrenado — para concentrar el esfuerzo en la integración de la percepción con el control y en su medición sobre el robot físico.

Las contribuciones principales son: (i) una arquitectura de dos niveles, con una frontera explícita entre el cómputo que decide y el que actúa, y un freno automático independiente de la capa de decisión; (ii) una máquina de estados con arbitraje de conflictos, verificable por completo sin hardware; (iii) una calibración de la tracción por rueda que compensa dos motores con características distintas sin usar codificadores; y (iv) un registro ciclo por ciclo de cada corrida, que permite reconstruir cada decisión y que sostiene los resultados que aquí se informan.

El resto del informe se organiza así: la Sección 2 describe el diseño y la implementación; la Sección 3, la metodología de evaluación; la Sección 4 presenta y discute los resultados, y la Sección 5 resume las conclusiones y el trabajo futuro.

1.1 Planteamiento del problema y objetivos

El escenario es un ambiente interior de piso liso y despejado, de unos pocos metros de lado. En él se colocan dos objetos: un *objetivo*, que es un oso de peluche impreso en una hoja A4 (unos 20 cm de ancho), y una *amenaza*, que es una mochila real. El robot arranca en una posición y orientación cualesquiera y no conoce la ubicación de los objetos.

Las restricciones son las siguientes. Todo el cómputo se realiza a bordo, sin red ni teleoperación; la inferencia corre en la CPU de una Raspberry Pi 5, sin acelerador; los motores son de corriente continua con reductora, sin lazo de velocidad; la cámara es una cámara web USB fija, a unos 10 cm del piso, y el robot se alimenta con baterías.

1.1.1 Objetivo general

Diseñar, construir y evaluar un robot móvil autónomo que, mediante detección de objetos con una red neuronal ejecutada a bordo, busque un objeto objetivo, se aproxime a él y se detenga a una distancia definida, y que evada un objeto amenaza, dando prioridad a la evasión cuando ambos están a la vista.

1.1.2 Objetivos específicos

- Ejecutar la detección de objetos en la Raspberry Pi 5 a no menos de 5 imágenes por segundo dentro del lazo completo de percepción, decisión y actuación.
- Implementar una máquina de estados con las conductas de búsqueda, aproximación, llegada y evasión, y con un arbitraje explícito que dé prioridad a la evasión.
- Diseñar una ley de control que centre el objetivo en la imagen y detenga el robot a una distancia definida, usando sólo la caja delimitadora que entrega la red.
- Desarrollar el control de bajo nivel en un microcontrolador, con compensación de las diferencias entre motores y un mecanismo de parada segura ante la pérdida de comunicación.
- Incorporar un sensor de distancia para evitar el avance contra obstáculos y una señal sonora de los eventos.
- Medir el desempeño del sistema en el robot físico: velocidad de procesamiento, latencias, alcance de detección, tiempos y éxito de cada conducta.

1.2 Fundamentos y trabajos relacionados

Detección de objetos. Los detectores de una etapa de la familia YOLO plantean la detección como una única regresión desde la imagen hacia cajas delimitadoras y probabilidades de clase, lo que los hace aptos para tiempo real [1]. YOLOv8 [3] es una versión sin anclas de esa familia, con variantes de distinto tamaño; la más chica (*nano*, YOLOv8n) tiene unos 3 millones de parámetros [2]. Los pesos que se distribuyen están entrenados en COCO [4], un conjunto con 80 clases de objetos cotidianos fotografiados en su contexto habitual. Esta última característica resultó relevante: las mochilas de COCO aparecen en su mayoría sobre la espalda de una persona y vistas a su altura, no apoyadas en el piso y vistas desde abajo (Sección 4.2).

Robótica basada en comportamientos. La idea de que conductas simples y reactivas, conectadas casi directamente de los sensores a los motores, producen un comportamiento aparentemente dirigido aparece en los vehículos de Braitenberg [5], cuyos casos canónicos son justamente la atracción y la huida. La arquitectura de subsumción [6] organiza esas conductas en capas, donde las de mayor prioridad inhiben a las demás; el arbitraje por prioridad fija entre conductas es el esquema más simple de coordinación en este paradigma [7]. En este trabajo la evasión subsume a la aproximación.

Control visual. El control basado en imagen regula directamente características medidas en el plano de la imagen, sin reconstruir la posición del objeto en el espacio [8]. Aquí se usan dos características de la caja delimitadora: su posición horizontal, para el giro, y su área, para el avance.

Locomoción diferencial. El modelo cinemático de un robot de dos ruedas motrices independientes relaciona las velocidades de las ruedas con la velocidad lineal y la angular del vehículo [9]; es el modelo que usa el microcontrolador para mezclar las consignas.

Respecto de las soluciones habituales, que ejecutan la inferencia en una GPU embebida o en una computadora externa, o que reentrenan la red para sus propios objetos, esta propuesta se limita a un modelo preentrenado y a la CPU de la Raspberry

Pi. El aporte no está en la red, sino en su integración en un lazo de control físico y en la medición de ese lazo.

2 Diseño e implementación del sistema

2.1 Arquitectura general

El sistema tiene dos niveles (Figura 1). El nivel alto corre en la Raspberry Pi 5: captura una imagen, ejecuta la red, actualiza la máquina de estados y calcula una consigna de movimiento. El nivel bajo corre en un ESP32: recibe la consigna por un enlace serie, la convierte en una señal de potencia para cada rueda, lee el sensor ultrasónico, maneja el zumbador y devuelve telemetría.

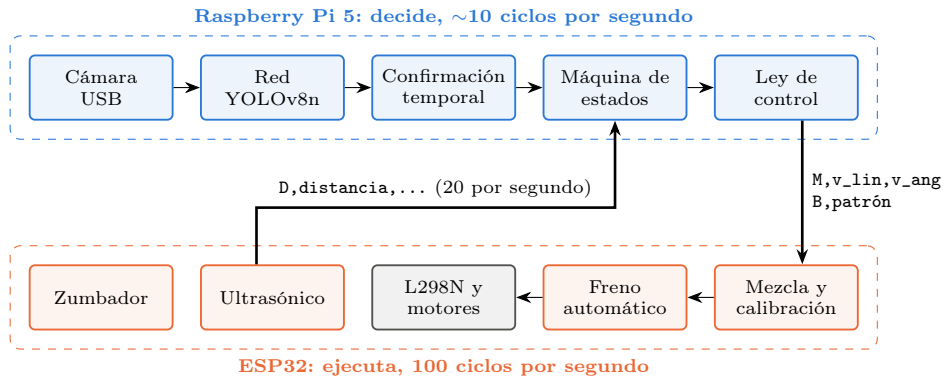


Figura 1 Arquitectura general. La Raspberry Pi nunca maneja un motor y el ESP32 nunca toma una decisión de comportamiento. Entre ambos viaja una consigna de intención (avance y giro, de -100 a 100) y vuelve la distancia medida.

Las decisiones de diseño que definen esta arquitectura son las siguientes.

- **Un solo microcontrolador de bajo nivel.** Se descartó combinar dos microcontroladores: duplicaba el firmware y los protocolos sin aportar funcionalidad.
- **Modelo preentrenado, sin entrenamiento propio.** El ciclo de captura, etiquetado y entrenamiento añadía riesgo de cronograma sin ser el aporte buscado.
- **La compensación de los motores vive en el ESP32.** El nivel alto envía una intención normalizada; sólo el microcontrolador conoce los valores de modulación de cada rueda. Así la ley de control no depende del hardware de tracción.
- **Las capas que deciden no acceden al hardware.** La máquina de estados y la ley de control reciben percepciones y un instante de tiempo como argumentos y devuelven una consigna; por eso pueden ejecutarse a velocidad de simulación y verificarse sin cámara ni robot.
- **Confirmación temporal.** Una detección aislada no cambia el estado: una clase se da por presente tras 3 imágenes consecutivas y por perdida tras 5 sin ella.

- **Parada segura en dos niveles.** Si el lazo de visión se retrasa más de 0,40 s, la Raspberry Pi envía una consigna nula; si el ESP32 no recibe consignas durante 0,5 s, frena los motores por su cuenta.

2.2 Componentes e implementación

2.2.1 Plataforma y electrónica

La Tabla 1 resume los componentes y la Figura 2 muestra el robot terminado. La alimentación usa dos fuentes independientes con masa común: una batería portátil de 5 V para la Raspberry Pi (y, por sus puertos USB, la cámara y el ESP32) y una fuente de 12 V sólo para la etapa de potencia de los motores. La separación no fue una decisión inicial sino el resultado de una medición (Sección 4).

Tabla 1 Componentes del robot.

Componente	Detalle	Función
Raspberry Pi 5 ESP32	8 GB de RAM, sin monitor Placa de desarrollo con puente USB-serie	Percepción y decisión Control de bajo nivel y seguridad
Etap. de potencia Motores (2) Chasis	Módulo L298N, puente H doble [10] GM25-370, 12 V, con reductora Dos ruedas de 60 mm y una rueda loca	Accionamiento de los motores Tracción diferencial Locomoción
Cámara	Cámara web USB, 640×480, MJPEG	Detección de objetos
Ultrasónico	Parallax PING))) , un pin de señal [11]	Distancia frontal, 2 cm a 3 m
Zumbador	Activo, de dos terminales	Señalización de eventos
Batería portátil	20 000 mAh, 5 V / 3 A	Alimenta la Raspberry Pi
Fuente de 12 V	10 000 mAh, con salida de 12 V	Alimenta sólo los motores

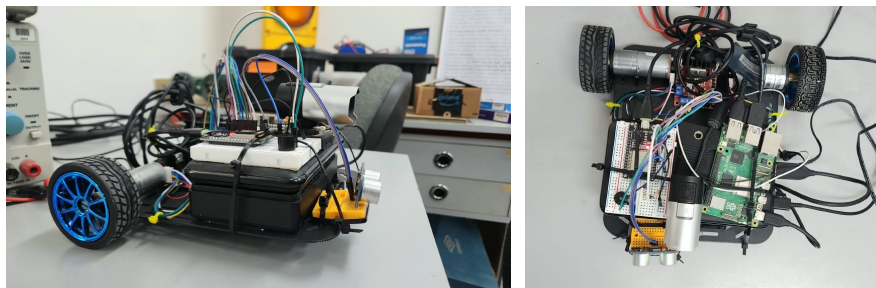


Figura 2 El robot terminado. Al frente, el sensor ultrasónico y, sobre él, la cámara; en la placa de pruebas, el ESP32 y el zumbador; a su lado, la Raspberry Pi 5; atrás, la etapa de potencia y los motores.

El sensor ultrasónico usa una sola línea para el disparo y para el eco, y la entrega a 5 V; como las entradas del ESP32 admiten 3,3 V [12], se intercala un divisor resistivo (1 k Ω en serie y 2 k Ω a masa). El firmware configura el pin como salida para emitir el pulso de disparo y de inmediato como entrada; una interrupción mide el ancho del pulso de eco, de modo que la medición no bloquea el lazo de control.

2.2.2 Control de bajo nivel y calibración de la tracción

El ESP32 recibe la consigna (v_{lin}, v_{ang}) , con valores enteros entre -100 y 100 , y la mezcla según el modelo diferencial [9]:

$$u_{izq} = v_{lin} + v_{ang}, \quad u_{der} = v_{lin} - v_{ang}, \quad (1)$$

donde $v_{ang} > 0$ significa girar a la derecha. Si alguna rueda supera el rango, se escala el par completo para conservar la curvatura pedida.

Los dos motores, aunque del mismo modelo, resultaron distintos: una rueda entrega 2,11 veces más velocidad que la otra por unidad de ciclo de trabajo, y sus umbrales de arranque también difieren. Como no hay codificadores, la compensación se hace a lazo abierto con una recta por rueda. Para un comando normalizado $u \in (0, 1]$, el ciclo de trabajo aplicado es

$$d(u) = d_{\min} + u(d_{\max} - d_{\min}), \quad (2)$$

con un piso $d_{\min}$ y un techo $d_{\max}$ propios de cada rueda, medidos de forma que ambas giren a la misma velocidad para el mismo u (Figura 3). Al pasar de reposo a movimiento se aplica un pulso breve de mayor potencia para vencer la fricción estática. La modulación es de 1 kHz y 10 bits.

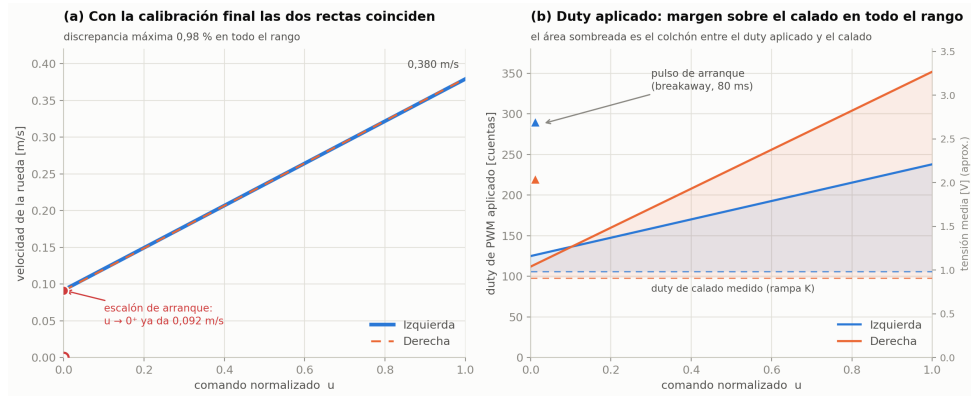


Figura 3 Calibración de la tracción con las ruedas en el aire. (a) Con una recta por rueda, las velocidades coinciden con una discrepancia máxima de 0,98 %. (b) Ciclo de trabajo aplicado a cada rueda. Con el robot apoyado hubo que elevar pisos y techos (Tabla A1).

2.2.3 Percepción

La red es YOLOv8n con los pesos de COCO, ejecutada con imagen de entrada de 320 píxeles. Las clases de interés se declaran por nombre: **teddy bear** para el objetivo y **backpack** para la amenaza. De cada detección se conservan la confianza y dos magnitudes normalizadas: el error lateral $e_x \in [-1, 1]$, que es la posición horizontal del centro de la caja respecto del centro de la imagen, y el área relativa $a \in [0, 1]$, que es el área de la caja dividida por la de la imagen.

En el piso se añadieron dos ajustes. Primero, la mochila real, apoyada en el piso y vista desde 10 cm de altura, es clasificada por el modelo como **suitcase** y no como **backpack**; se agregó un alias que asimila ambas clases a la amenaza. Segundo, a 1,2 m la confianza de esa detección vale entre 0,32 y 0,41, en el borde del umbral general de 0,35; la amenaza usa un umbral propio de 0,20.

La cámara opera en MJPEG. Cada ciclo se calcula además una medida de la integridad de la imagen: la fracción de filas ocupadas por franjas grises uniformes, que es la huella de un paquete USB perdido en ese formato.

2.2.4 Máquina de estados

El comportamiento es una máquina de cuatro estados (Figura 4).

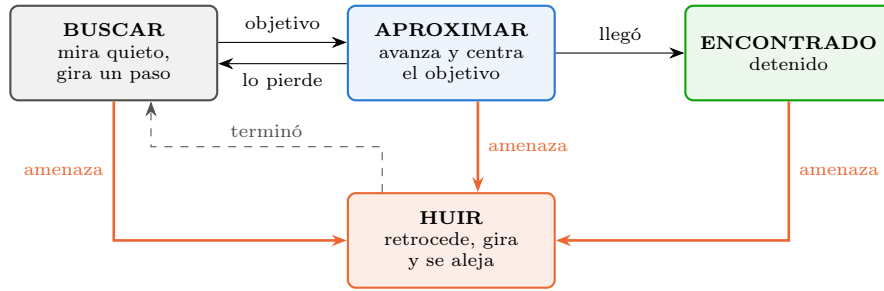


Figura 4 Máquina de estados. La amenaza interrumpe cualquier estado, salvo una evasión en curso.

BUSCAR. El robot barre el entorno a pasos: permanece quieto 0,6 s y gira durante 0,25 s (unos 18°). El barrido continuo se descartó porque, girando, la imagen sale movida y la red no detecta nada. Si aparece una detección aún no confirmada, el robot se queda quieto hasta confirmarla o descartarla. Tras 17 s de barrido, equivalentes a una vuelta, avanza 1 s para cambiar el punto de vista.

APROXIMAR. Con el objetivo confirmado se aplica la ley de control de la Sección 2.2.5. Si el objetivo se pierde, el barrido se reanuda hacia el lado donde se lo vio por última vez.

ENCONTRADO. El robot se detiene y emite un tono. Es un estado terminal, salvo que aparezca la amenaza.

HUIR. Es una maniobra de tres fases cronometradas: retroceso de 1,5 s girando hacia el lado opuesto a la amenaza, giro en el lugar de 1,2 s en el mismo sentido y avance de 2,5 s. No se reevalúa durante su ejecución. Al terminar hay un período de 2 s en el que la amenaza se ignora; sin él, la persistencia de la confirmación temporal

encadenaba una segunda evasión idéntica, defecto que se detectó en la verificación sin hardware antes de que el robot se moviera.

Uso del sensor ultrasónico. La distancia frontal interviene con dos umbrales. En APROXIMAR, una lectura de 20 cm o menos se interpreta como llegada si el objetivo está a la vista, razonablemente centrado ($|e_x| < 0,7$) y ya ocupa el 60 % del área de parada; en caso contrario se trata como un obstáculo ajeno y el robot se desvía. En la tercera fase de HUIR y en el avance de BUSCAR, que son los dos tramos en que el robot avanza sin un objetivo a la vista, una lectura de 30 cm o menos hace que gire en el lugar en vez de avanzar. Un obstáculo se sostiene durante 0,3 s después de la última lectura que lo detectó. Una lectura ausente se trata como “desconocido”, nunca como distancia nula: sin sensor, el robot se comporta como antes de incorporarlo.

2.2.5 Ley de control

La aproximación usa dos lazos proporcionales sobre la imagen:

$$v_{ang} = \text{sat}(100 K_{\omega} e_x, \pm v_{ang}^{\text{máx}}), \quad v_{ang} = 0 \text{ si } |e_x| < \epsilon, \quad (3)$$

$$v_{lin} = \text{sat}(100 K_v (a_{obj} - a), [0, v_{lin}^{\text{máx}}]), \quad (4)$$

con la restricción adicional

$$|v_{ang}| \leq 0,7 v_{lin}. \quad (5)$$

El avance decrece a medida que la caja crece y se anula cuando su área alcanza a_{obj} . Como el área aparente varía con el inverso del cuadrado de la distancia, $a \propto 1/d^2$, el valor de a_{obj} fija la distancia de parada para un objeto de tamaño dado; no es una medición de distancia. La restricción (5) garantiza que, avanzando, la rueda interior nunca se detenga ni invierta su marcha. Es necesaria porque la tracción no tiene arranque suave: la velocidad mínima de una rueda es 0,09 m/s, de modo que una rueda que cruza por cero produce un pívoteo brusco justo al llegar. El Listado 1 muestra su implementación.

```

1 def perseguir(deteccion, **kw):
2     """(v_lin, v_ang) para ir hacia una deteccion."""
3     v_lin = avanzar_hacia(deteccion.area_ratio) # ec. (4)
4     v_ang = girar_hacia(deteccion.error_x)      # ec. (3)
5     if v_lin > 0:
6         tope = int(0.7 * v_lin)                 # ec. (5)
7         v_ang = acotar(v_ang, -tope, tope)
8     return v_lin, v_ang

```

Listado 1 Núcleo de la ley de control (versión resumida de control.py).

2.2.6 Protocolo, software y versiones

El enlace entre los dos niveles es una línea serie a 115 200 baudios con mensajes de texto, uno por línea: M,v_lin,v_ang (movimiento) y B,patrón (zumbador) hacia el

ESP32, y `D, dist, enc_izq, enc_der` (telemetría, 20 por segundo) hacia la Raspberry Pi. El formato de texto se eligió porque permite depurar con un monitor serie.

El software de alto nivel está escrito en Python y se divide en módulos con una sola responsabilidad: `vision.py` (captura, inferencia y confirmación temporal), `estados.py` (máquina de estados), `control.py` (ley de control), `enlace_esp32.py` (protocolo y parada segura), `config.py` (todos los parámetros, cada uno con el origen de su valor), `registro.py` (registro de cada corrida) y `main.py` (lazo principal). El firmware está escrito en C++ sobre el entorno Arduino. La Tabla 2 lista las versiones empleadas.

Tabla 2 Software y versiones.

Elemento	Versión
Sistema operativo de la Raspberry Pi	Raspberry Pi OS (Debian 13), 64 bits
Python	3.13.5
Ultralytics (YOLOv8)	8.4.145, modelo <code>yolov8n.pt</code>
PyTorch / torchvision	2.14.0 / 0.29 (sólo CPU)
OpenCV	5.0.0
Firmware del ESP32	PlatformIO, núcleo Arduino para ESP32 2.0.17

3 Metodología experimental

El desarrollo y la evaluación siguieron cuatro etapas, cada una diseñada para probar una capa antes de depender de ella.

1. Banco de pruebas. Con el robot elevado se verificaron el cableado, el protocolo y la parada segura, y se calibró la tracción. Los umbrales de arranque se midieron con rampas descendentes del ciclo de trabajo, y la velocidad de cada rueda, contando vueltas en ventanas cronometradas.

2. Verificación sin hardware. Un programa (`prueba_estados.py`) ejecuta la máquina de estados real con un reloj simulado y percepciones sintéticas, y comprueba 79 propiedades del comportamiento: transiciones, prioridad de la evasión, orden y duración de las fases, rango de las consignas, tratamiento del sensor ausente, entre otras. Se ejecuta en la computadora de desarrollo y en la Raspberry Pi antes de cada sesión. Un simulador cinemático complementario ejecuta la misma máquina contra un robot simulado, con retardo y pérdida de detecciones.

3. Medición de la visión. Se midió la velocidad de procesamiento con la cámara real para cuatro tamaños de imagen, y el alcance de detección con el objetivo a distancias crecientes.

4. Pruebas en el piso. El robot se evaluó suelto en tres jornadas (30 de setiembre, 1 y 2 de octubre de 2026); la primera se dedicó a la tracción y a la cámara, y las corridas de comportamiento corresponden a las otras dos. Cada corrida tiene una duración máxima fijada de antemano y deja un registro con una fila por ciclo del lazo: instante, estado, consigna enviada, clases confirmadas, confianza, e_x y a de cada

objeto, distancia del ultrasónico, integridad de la imagen y el motivo de la decisión en texto. Un segundo archivo guarda los parámetros vigentes. Un programa de análisis extrae de esos registros las métricas de la sección siguiente. Las corridas que mueven motores se anuncian antes de lanzarse, y la alimentación de los motores queda al alcance del operador como corte de emergencia.

Los escenarios de prueba fueron cuatro: (A) aproximación, con el objetivo a 1–2 m en distintas direcciones; (B) evasión, con la amenaza a 1 m de frente; (C) evasión hacia un obstáculo, con una pared a 0,8–1 m detrás del robot; y (D) demostración completa, con ambos objetos.

Alcance de la evaluación. Ninguna de las 24 corridas realizadas en el piso forma parte de una serie con condiciones fijas. Los resultados deben leerse como evidencia de funcionamiento y como mediciones de tiempos y distancias, no como tasas de éxito estadísticamente representativas.

3.1 Métricas de evaluación

La *velocidad del lazo* es el número de ciclos completos (captura, inferencia, decisión y envío) por segundo. La *latencia de decisión* es el tiempo entre la exposición de la imagen y el envío de la consigna. El *tiempo de confirmación* y la *latencia de reacción* se miden desde la primera imagen que contiene el objeto hasta el cambio de estado. El *tiempo hasta encontrar* se mide desde el inicio de la corrida hasta el estado ENCONTRADO. La *distancia de parada* se midió con cinta métrica y, desde su incorporación, con el sensor ultrasónico.

La tasa de éxito de una conducta sobre N corridas de un escenario es

$$E = \frac{1}{N} \sum_{i=1}^N \mathbf{1} [\text{la corrida } i \text{ completa la conducta sin colisión}], \quad (6)$$

y la integridad de la imagen se cuantifica, para una corrida de M imágenes, como la fracción de imágenes dañadas

$$F = \frac{1}{M} \sum_{j=1}^M \mathbf{1} [r_j > 0,01], \quad (7)$$

donde r_j es la fracción de filas de la imagen j ocupadas por franjas uniformes.

3.2 Configuración de las pruebas

Las pruebas se hicieron en interiores, sobre piso liso, con iluminación artificial. El objetivo se fijó en posición vertical a la altura de la cámara, y la amenaza se apoyó en el piso. La Tabla 3 reúne los parámetros con los que se obtuvieron los resultados finales; el Apéndice A detalla la calibración de la tracción y cada corrida.

Tabla 3 Parámetros principales de la evaluación.

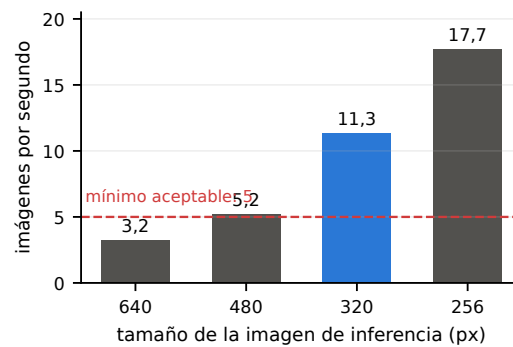
Parámetro	Valor	Unidad / descripción
Tamaño de la imagen de inferencia	320	píxeles
Umbral de confianza (objetivo / amenaza)	0,35 / 0,20	—
Confirmar / dar por perdido	3 / 5	imágenes consecutivas
Ganancia de giro K_ω	0,5	—
Ganancia de avance K_v	2,4	—
Área de parada a_{obj}	0,35	fracción de la imagen
Zona muerta ϵ	0,08	de ϵ_x
Consigna máxima de avance / giro	55 / 60	de 100 ($55 \approx 0,25$ m/s)
Paso de búsqueda: mirar / girar	0,6 / 0,25	s (giro de $\sim 18^\circ$)
Evasión: retroceso / giro / avance	1,5 / 1,2 / 2,5	s
Período sin reevaluar la amenaza	2,0	s
Ultrasónico: llegada / esquivar	20 / 30	cm
Retención del obstáculo	0,3	s
Vigencia de la consigna / parada del ESP32	0,40 / 0,50	s
Frecuencia del lazo de bajo nivel	100	Hz
Duración de cada corrida	12 a 90	s, fijada de antemano

4 Resultados y discusión

4.1 Resultados

4.1.1 Velocidad de procesamiento y alimentación

La Figura 5 muestra la velocidad medida con la cámara real para cuatro tamaños de imagen. Con 640 píxeles la Raspberry Pi 5 procesa 3,2 imágenes por segundo, por debajo del mínimo fijado; con 320 píxeles alcanza 11,3. Se adoptó 320: es el mayor tamaño que supera el mínimo con margen, y su costo es el alcance de detección. La inferencia toma unos 88 ms y la imagen llega con 36 a 64 ms de antigüedad, lo que da una latencia de decisión de unos 150 ms; a la velocidad máxima usada (0,25 m/s) equivale a unos 4 cm de avance.

**Figura 5** Velocidad de procesamiento en la Raspberry Pi 5 según el tamaño de la imagen de inferencia, incluyendo la captura.

Con el lazo completo y el enlace serie real se midieron 11,2 ciclos por segundo en banco. En el piso, registrando además la integridad de la imagen, la velocidad fue de 10,5 a 10,7 ciclos por segundo, y de 9,6 a 10,0 al grabar video; en las corridas de operación independiente estuvo entre 9,1 y 10,4. El percentil 95 del período del lazo estuvo entre 97 y 136 ms.

Con una sola fuente para todo el robot, la tensión de 5 V caía durante la inferencia: el lazo bajaba a 8,8 ciclos por segundo y la Raspberry Pi recortaba su frecuencia el 43 % del tiempo. Con dos fuentes separadas la velocidad fue de 11,1 ciclos por segundo, sin ningún evento de baja tensión en 168 muestras, y la tensión se mantuvo entre 4,91 y 5,13 V. En 150 s de inferencia continua la temperatura se estabilizó entre 73 y 75 °C sin reducción de frecuencia.

4.1.2 Aproximación al objetivo

La Figura 6 (arriba) muestra la secuencia vista por la cámara del robot, y la Figura 7, las variables registradas durante una aproximación. El objetivo aparece en el borde derecho de la imagen ($e_x = +0,63$); el robot avanza a la consigna máxima mientras gira, el error lateral cruza la zona muerta en 1,3 s, el avance decrece a medida que crece el área y el robot decide detenerse cuando a alcanza 0,35, a los 2,1 s. En ese instante el ultrasónico marca 28 cm; por inercia y por la latencia, el robot queda detenido a 22 cm.

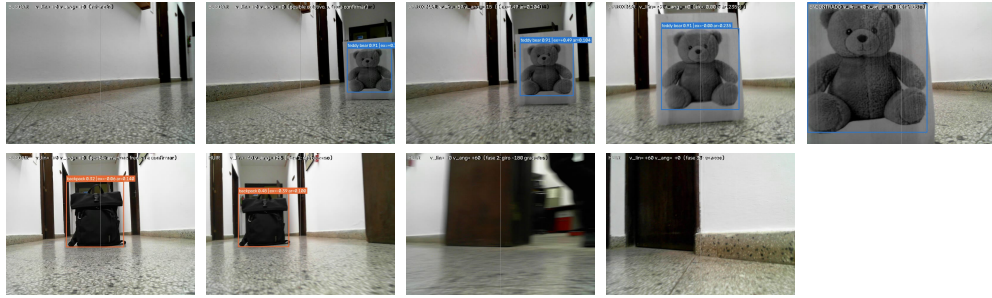


Figura 6 Imágenes de la cámara del robot con las detecciones superpuestas. Arriba, aproximación: búsqueda, primera detección, avance, centrado y detención. Abajo, evasión: detección de la amenaza, retroceso, giro y alejamiento.

El objetivo se confirmó típicamente 0,2 s después de la primera imagen en que apareció (entre 0,11 y 0,40 s en las corridas registradas). El tiempo de aproximación, desde la confirmación hasta la detención, fue de 1,7 a 3,1 s partiendo de alrededor de 1 m. El tiempo total hasta encontrar depende de la orientación inicial: 3,1 s con el objetivo de frente, y 15,0 y 17,0 s con el objetivo a 1 m y a 90°, porque el barrido a pasos tarda unos 17 s por vuelta. El objetivo impreso se detectó hasta 1,3 m, aproximadamente; ubicado a 2 m, el robot lo encontró recién a los 65 s, cuando los avances de la búsqueda lo acercaron.

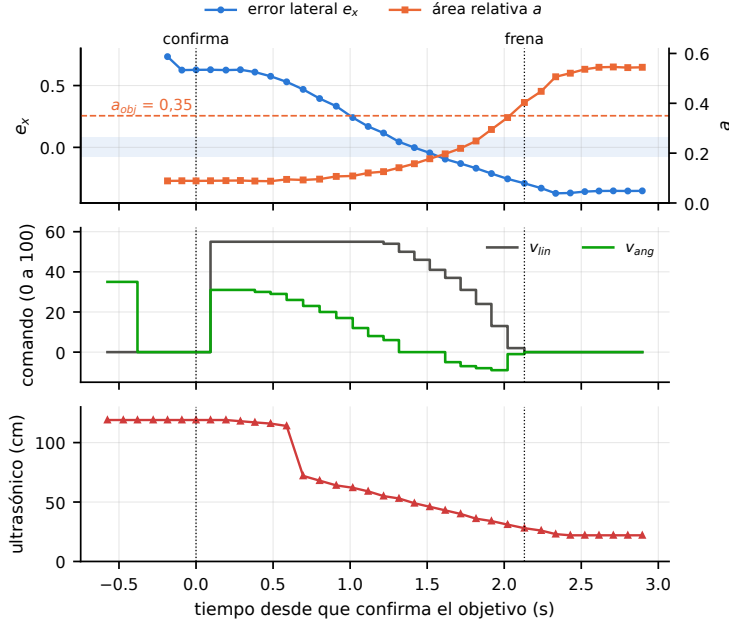


Figura 7 Una aproximación registrada (2 de octubre). Arriba: error lateral y área relativa del objetivo; la franja sombreada es la zona muerta del giro. Centro: consignas enviadas. Abajo: distancia del sensor ultrasónico, que al principio apunta a otra superficie.

4.1.3 Evasión y sensor ultrasónico

El robot decidió huir típicamente 0,2 s después de la primera imagen que contenía la amenaza (entre 0,18 y 0,27 s cuando las tres imágenes que la confirman son consecutivas). La maniobra completa dura 5,2 s. En la demostración con ambos objetos, la amenaza colocada junto al objetivo durante una aproximación interrumpió esta última, como exige la prioridad.

La Tabla 4 resume las mediciones del sensor ultrasónico y la Figura 8 muestra dos evasiones hacia una pared. Con un único umbral de 20 cm el robot interrumpió el avance al leer 18 cm, pero como en ese momento no frena sino que gira en el lugar, la lectura mínima bajó hasta 11 cm. Con un umbral de esquivar propio de 30 cm, el avance se interrumpió a 29 cm y la lectura no bajó de 27 cm. En el panel (b) se observa además una limitación: al terminar el giro de la segunda fase, que se hace sin información de distancia, el robot ya estaba a 7 cm de un obstáculo.

Las corridas de operación independiente, hechas en un espacio más reducido, mostraron los límites del sensor. De nueve interrupciones del avance, cinco ocurrieron al leer 29 o 30 cm y cuatro recién al leer entre 16 y 20 cm, porque el obstáculo apareció de un ciclo al siguiente, como ocurre con una superficie abordada de forma oblicua. Y durante el giro de esquivar la lectura mínima llegó a 3 cm en dos corridas, lo que indica que el robot quedó prácticamente en contacto con el obstáculo.

Tabla 4 Mediciones con el sensor ultrasónico.

Prueba	Resultado
Robot quieto, caja a 30 cm	30 cm, 120 lecturas válidas de 120
Robot quieto en el piso, pared a 1,5 m	153 cm, estable; sin ecos del piso
Avance a consigna 40 hasta leer ≤ 20 cm	orden de frenar a 19 cm; detenido a 16 cm
Evasión a consigna 60, umbral único de 20 cm	interrumpe a 17–18 cm; mínimo 11–16 cm
Evasión a consigna 60, umbral de esquite 30 cm	interrumpe a 29 cm; mínimo 27 cm
Ídem, en operación independiente (9 casos)	interrumpe a 29–30 cm (5) o a 16–20 cm (4); mínimo 3 cm al girar
Aproximación al objetivo	sin desvíos falsos; detenido a 22 cm

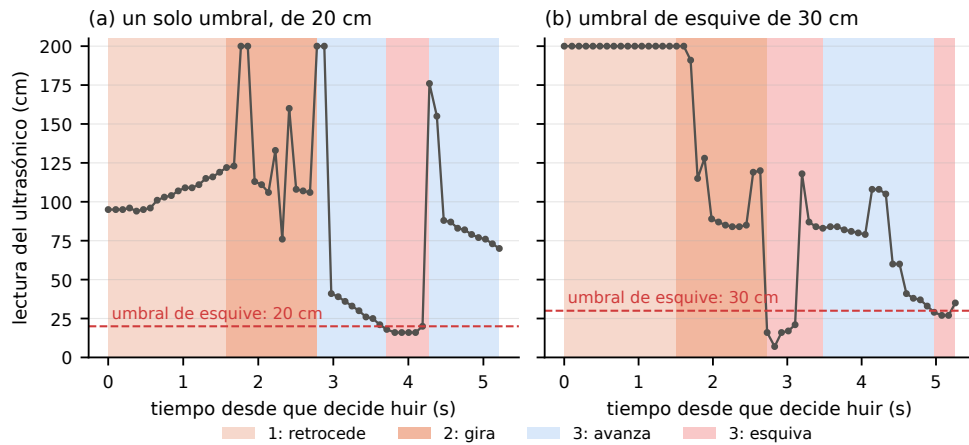


Figura 8 Lectura del sensor ultrasónico durante dos evasiones hacia una pared. El fondo indica la fase de la maniobra. Las lecturas mayores de 2 m, que corresponden a “sin obstáculo”, se muestran recortadas en 200 cm.

4.1.4 Resumen de métricas y de corridas

La Tabla 5 compara las métricas con los objetivos fijados antes de las pruebas, y la Tabla A2 del Apéndice A detalla las 24 corridas en el piso.

De las 10 corridas de puesta a punto cuyo escenario incluía el objetivo, 8 terminaron en ENCONTRADO. Las dos restantes no son fallas de la conducta: en una la cámara entregaba imágenes dañadas y la red no detectó nada, y la otra se interrumpió a los 52 s al advertirse que el sensor ultrasónico estaba desconectado. Las colisiones no se instrumentaron: con el umbral de esquite de 30 cm, el registro no muestra lecturas inferiores a 27 cm *durante el avance* en ninguna corrida; antes de incorporar el sensor, el operador observó una evasión que terminó contra una pared.

En las corridas de operación independiente el resultado fue distinto (Tabla 6). En las tres primeras, con la amenaza a la vista y poco espacio libre, el robot no llegó al objetivo, y en la última de ellas encadenó seis evasiones en 60 s: cuatro comenzaron unos 2 s después de terminar la anterior, que es la duración del período en que la amenaza se ignora. Tras modificar el sentido de la búsqueda posterior a la evasión, el

Tabla 5 Métricas medidas frente a los objetivos definidos en el plan.

Métrica	Objetivo	Medido
Velocidad del lazo completo	≥ 5 por segundo	9,1 a 10,7 según la corrida
Latencia de decisión	registrar	~ 150 ms
Alcance de detección del objetivo	hasta 2 m	$\sim 1,3$ m (no cumple)
Tiempo de confirmación	registrar	$\sim 0,2$ s (0,11 a 0,40 s)
Latencia de reacción ante la amenaza	registrar	$\sim 0,2$ s (0,18 a 0,27 s)
Tiempo hasta encontrar	registrar	3,1 s de frente; 15 a 17 s a 90° ; 65 s a 2 m
Distancia de parada	~ 30 cm	~ 30 cm con cinta; 22 cm según el ultrasónico
Éxito de aproximación	≥ 8 de 10	8 de 10 de puesta a punto; en operación independiente, 0 de 3 antes y 3 de 5 después del cambio en la búsqueda
Éxito de evasión	≥ 8 de 10	39 maniobras iniciadas; 2 corridas iniciales sin disparo (corregido); sin serie verificada sin hardware; 1 caso en el piso
Prioridad de la evasión	10 de 10	0,5 s, verificada en hardware
Parada ante pérdida de órdenes	< 1 s	
Imágenes dañadas F	0	episodios el 1 de octubre; el 2 de octubre, 0 de 3675 a la mañana y 33 de 3793 a la tarde

robot llegó al objetivo en tres de cinco corridas y sólo una de quince evasiones comenzó apenas terminada la anterior. En las otras dos corridas siguió huyendo cuatro y cinco veces en un minuto, ahora con intervalos de 3 a 16 s: completaba parte del barrido, volvía a encontrar la amenaza y no llegó a detectar el objetivo. Las condiciones de estas corridas no se controlaron, por lo que la comparación es indicativa.

Tabla 6 Corridas de operación independiente, antes y después de modificar el sentido de la búsqueda posterior a la evasión.

	Antes	Después
Corridas	3	5
Llegaron al objetivo	0	3
Evasiones iniciadas	8	15
Evasiones iniciadas menos de 2,5 s después de terminar la anterior	4	1

4.1.5 Integridad de la imagen

El 1 de octubre, con el robot quieto, 69 de 71 imágenes llegaron con franjas grises, y la red no detectó el objetivo ubicado a 1 m (Figura 9). El episodio se repitió de forma intermitente con los motores detenidos e incluso sin alimentación de motores, y cesó sin intervención: en las ocho corridas siguientes no hubo ninguna imagen dañada en 2104. El registro del sistema operativo mostró que se trataba de paquetes USB perdidos. El 2 de octubre, tras reinsertar el conector de la cámara —que se había

soltado— y con las baterías recién cargadas, no hubo imágenes dañadas en las 3675 de la mañana. Como ambas condiciones cambiaron a la vez, la causa no quedó aislada. Esa misma tarde las franjas reaparecieron en dos de las ocho corridas de operación independiente (24 de 569 imágenes en una y 9 de 600 en otra), lo que confirma que el problema sigue presente.

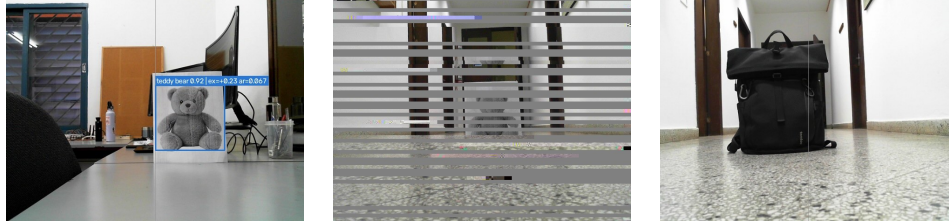


Figura 9 Izquierda: imagen íntegra, con el objetivo detectado. Centro: imagen dañada por pérdida de paquetes USB, con el robot quieto; el objetivo está detrás de las franjas. Derecha: la mochila vista desde el robot, que el modelo clasifica como *suitcase*.

4.2 Discusión y limitaciones

Cumplimiento de los objetivos. El robot realiza las conductas previstas con todo el cómputo a bordo y con más del doble de la velocidad de procesamiento mínima. La arquitectura de dos niveles cumplió su función: ningún cambio en la calibración de los motores obligó a modificar la ley de control, y la parada segura actuó cada vez que el programa de alto nivel se detuvo.

El contexto de validez de las mediciones. El hallazgo más repetido del trabajo es que un valor medido deja de ser válido fuera de las condiciones en que se obtuvo. La calibración de la tracción, exacta al 1 % con las ruedas en el aire, no movía una de las ruedas con el robot apoyado, porque le asignaba el menor ciclo de trabajo justamente a la rueda de menor par; hubo que recalibrar en el piso (Tabla A1). La velocidad de giro de $165^\circ/\text{s}$, medida en régimen sobre varias vueltas, produjo unos 110° en la fase de giro de la evasión, que dura alrededor de un segundo y parte de la marcha atrás. La etiqueta *backpack* es correcta para las imágenes con que se entrenó la red, pero la misma mochila vista desde 10 cm del piso es, para el modelo, una valija. Y la relación entre el giro de búsqueda y la detección, supuesta al diseñar, sólo se resolvió midiendo con la propia cámara cuánto se desplaza la escena en cada paso. La respuesta metodológica fue registrar junto a cada parámetro las condiciones en que se midió y probar una capa por vez.

Valor de la verificación sin hardware. El banco de verificaciones detectó defectos antes de que tuvieran consecuencias físicas, como el encadenamiento de evasiones por la persistencia de la confirmación temporal. También mostró su límite: una comprobación exigía que una detección no confirmada de la amenaza no detuviera el barrido, se cumplía siempre y era la causa de que el robot no huyera en el piso. Una verificación asegura que el código hace lo decidido, no que la decisión fuera correcta.

Limitaciones.

- *Evaluación.* No se completó la serie de 10 repeticiones por escenario; las tasas de la Tabla 5 provienen de pocas corridas con parámetros que cambiaron entre ellas.
- *Ciclo de evasiones.* Con la amenaza siempre a la vista y una pared cercana, el robot quedó en un ciclo: al huir encontraba la pared, giraba para esquivarla hasta quedar nuevamente orientado hacia la amenaza —en tres casos incluso avanzó hacia ella— y volvía a huir al terminar el período de 2 s. Se modificó el sentido del barrido posterior a la evasión: ahora es aleatorio, o contrario al lado de la amenaza si esta queda a la vista. En las cinco corridas posteriores el ciclo de 2 s prácticamente desapareció y el robot llegó al objetivo en tres, pero en las otras dos siguió huyendo de forma repetida sin encontrarlo. El giro de esquivar, que es la causa de fondo, no se modificó, y la comparación no proviene de una serie controlada.
- *Sensado de obstáculos.* El sensor sólo cubre el frente. El retroceso y el giro de la evasión se hacen sin información de distancia, y una pared abordada de forma muy oblicua no se detecta hasta estar a unos 20 cm, porque el eco se refleja en otra dirección: en espacios reducidos el robot llegó a quedar a 3 cm de un obstáculo mientras giraba. Los tres parámetros asociados al sensor se ajustaron con una corrida cada uno.
- *Alcance y dependencia del objeto.* El objetivo se detecta hasta 1,3 m con la imagen de 320 píxeles. La distancia de parada depende del tamaño del objeto con que se calibró.
- *Falsos positivos de la amenaza.* El umbral reducido a 0,20 produce detecciones aisladas sobre objetos oscuros grandes. La confirmación temporal impidió que desencadenaran una evasión en las pruebas, pero el resultado depende del entorno.
- *Tracción a lazo abierto.* Sin codificadores, el robot se desvía al acelerar y al frenar, y queda con el objetivo descentrado algunos grados al detenerse.
- *Integridad de la imagen.* La causa de la pérdida de paquetes USB no quedó aislada y el problema reapareció la última tarde; el sistema lo detecta y lo registra, pero no lo corrige.
- *Dependencia del enlace de operación.* Si se corta la sesión remota desde la que se lanza una corrida, el programa puede seguir ejecutándose hasta su límite de tiempo; la parada automática protege ante la caída del programa, no ante la del enlace.

5 Conclusiones

Se construyó un robot móvil que busca un objeto, se aproxima a él y se detiene a unos 30 cm, y que evade otro objeto con prioridad sobre la aproximación, con la detección ejecutada a bordo en una Raspberry Pi 5 a unas 10 imágenes por segundo. El objetivo general se cumplió en cuanto a la integración de percepción, decisión y actuación en el robot físico; la evaluación cuantitativa del éxito de cada conducta quedó incompleta, porque no se realizaron las series de repeticiones previstas.

Los resultados más relevantes son la latencia de decisión de unos 150 ms, la confirmación de un objeto y la reacción ante la amenaza en 0,2 s, la aproximación desde 1 m en 2 a 3 s y la interrupción del avance a 29 cm de un obstáculo frontal. La separación en dos niveles, la verificación del comportamiento sin hardware y el registro

ciclo por ciclo fueron las decisiones que más contribuyeron a llegar a ese resultado y a poder explicarlo.

Como trabajo futuro se propone: completar las series de 10 corridas por escenario, incluida una que evalúe de forma controlada el cambio en el sentido de búsqueda; acotar el giro de esquivé durante la evasión; aislar la causa de la pérdida de imágenes; añadir sensado de distancia lateral y trasero; incorporar codificadores para cerrar un lazo de velocidad por rueda; aumentar el alcance de detección usando una imagen mayor sólo mientras el robot está detenido en la búsqueda; y evaluar un modelo ajustado con imágenes tomadas desde la altura de la cámara del robot.

Contribución de los integrantes

Los dos integrantes participaron en la definición del problema y del alcance, en el montaje y el cableado del robot, en las decisiones de diseño, en las pruebas físicas y las observaciones en el piso, y en la revisión del software, los resultados y la documentación.

Disponibilidad del código y los datos

El código fuente, el firmware, los registros de las corridas, los videos y la documentación están disponibles en un repositorio público, bajo la licencia AGPL-3.0: <https://github.com/fedemoranf-alt/AguaraBot-RoboticaIA-MscIA>. El repositorio contiene: `src/` (software de la Raspberry Pi, incluido el banco de verificaciones), `firmware/` (firmware del ESP32, con su historial de versiones), `scripts_pi/` (puesta en marcha y operación), `logs/` (un archivo CSV y uno JSON por corrida), `media/` (videos tomados por la cámara del robot) y los documentos `HARDWARE.md` (análisis eléctrico y mediciones) y `GUIA_PRESENTACION.md` (guía de operación). Las verificaciones sin hardware se reproducen con `python prueba_estados.py` desde `src/`; las figuras de este informe, con `python generar_figuras.py`; y una corrida en el robot, con `demo.sh correr`, según la guía de operación.

Uso de herramientas de inteligencia artificial generativa

Durante todo el proyecto se utilizó un asistente basado en un modelo de lenguaje (Claude, de Anthropic, mediante la herramienta Claude Code). Se lo empleó para la asistencia en la programación del firmware y del software de alto nivel.

Referencias

- [1] Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788 (2016). <https://doi.org/10.1109/CVPR.2016.91>

- [2] Terven, J., Córdova-Esparza, D.-M., Romero-González, J.-A.: A comprehensive review of YOLO architectures in computer vision: from YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction* **5**(4), 1680–1716 (2023). <https://doi.org/10.3390/make5040083>
- [3] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLO (2023). <https://github.com/ultralytics/ultralytics>
- [4] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: common objects in context. In: *Computer Vision – ECCV 2014. Lecture Notes in Computer Science*, vol. 8693, pp. 740–755. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-10602-1_48
- [5] Braitenberg, V.: *Vehicles: Experiments in Synthetic Psychology*. MIT Press, Cambridge (1984)
- [6] Brooks, R.A.: A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation* **2**(1), 14–23 (1986). <https://doi.org/10.1109/JRA.1986.1087032>
- [7] Arkin, R.C.: *Behavior-Based Robotics*. MIT Press, Cambridge (1998)
- [8] Chaumette, F., Hutchinson, S.: Visual servo control. I. Basic approaches. *IEEE Robotics & Automation Magazine* **13**(4), 82–90 (2006). <https://doi.org/10.1109/MRA.2006.250573>
- [9] Siegwart, R., Nourbakhsh, I.R., Scaramuzza, D.: *Introduction to Autonomous Mobile Robots*, 2nd edn. MIT Press, Cambridge (2011)
- [10] STMicroelectronics: L298 – Dual full-bridge driver. Hoja de datos
- [11] Parallax Inc.: PING))) Ultrasonic Distance Sensor (#28015). Guía del producto
- [12] Espressif Systems: ESP32 Series Datasheet

Apéndice A Material complementario

A.1 Calibración de la tracción

La Tabla A1 compara la calibración obtenida con las ruedas en el aire con la que quedó cargada tras recalibrar con el robot apoyado. Los valores son cuentas del ciclo de trabajo, sobre un máximo de 1023.

Con la calibración en el aire, el rango útil de velocidad de cada rueda es de 0,091 a 0,380 m/s. La carga del robot desplaza ambas rectas hacia arriba sin cambiar su pendiente. Con la calibración de piso, un avance cronometrado recorrió 57 cm frente

Tabla A1 Calibración de la tracción, rueda izquierda / rueda derecha.

Parámetro	En el aire	En el piso
Piso de la recta $d_{\text{mín}}$	125 / 112	300 / 230
Techo de la recta $d_{\text{máx}}$	238 / 352	413 / 470
Pulso de arranque	290 / 220	380 / 280
Duración del pulso de arranque	80 ms	150 ms

a los 57 cm previstos. El giro en el lugar alcanza en régimen $124^\circ/\text{s}$ con una consigna de 35 y $165^\circ/\text{s}$ con una consigna de 60.

A.2 Corridas en el piso

La Tabla A2 lista las 24 corridas de comportamiento registradas en el piso, en orden cronológico. Las diez primeras corresponden al 1 de octubre, antes de incorporar el sensor ultrasónico; las seis siguientes, a la mañana del 2 de octubre, ya con el sensor y el zumbador; y las ocho últimas son las de operación independiente, de esa tarde: tres anteriores y cinco posteriores al cambio en el sentido de la búsqueda.

Tabla A2 Corridas en el piso. “Enc.” es el instante en que se alcanzó el estado ENCONTRADO; “c/s” son ciclos del lazo por segundo.

Día, hora	Escenario	s	c/s	Enc.	Resultado
1/10 15:21	Objetivo a 1 m, 90°	25	9,7	—	Sin detecciones: imagen dañada
1/10 16:02	Objetivo de frente, ~1 m	15	10,0	3,1 s	Llega; 37 cm con cinta (a_{obj} anterior)
1/10 16:23	Objetivo a 1 m, 90°	30	9,8	17,0 s	Llega; termina descentrado
1/10 16:29	Objetivo a 1 m, 90°	30	9,9	15,0 s	Llega, centrado
1/10 16:40	Amenaza a 1 m	12	10,0	—	No huye: el barrido no se detenía
1/10 16:43	Amenaza a 1 m	12	10,0	—	No huye: confianza bajo el umbral
1/10 16:50	Amenaza a 1 m	12	9,8	—	Huye; gira ~110°
1/10 16:54	Amenaza a 1 m	12	9,6	—	Evasión completa
1/10 17:07	Objetivo a ~2 m	90	9,9	65,3 s	Llega; lo pierde una vez
1/10 17:17	Objetivo y amenaza	90	9,9	85,4 s	4 evasiones; llega al final
2/10 10:34	Amenaza; pared a 1 m	30	10,6	29,3 s	1 evasión; el avance termina a 27 cm
2/10 10:36	Amenaza; pared a 0,8 m	30	10,7	—	3 evasiones; esquivo a 18 cm
2/10 10:41	Objetivo a 1,5–2 m	60	10,5	58,8 s	2 evasiones; llega
2/10 10:54	Objetivo y amenaza	52	10,7	—	Sensor desconectado; interrumpida
2/10 10:59	Objetivo y amenaza	90	10,7	21,0 s	2 evasiones; esquivo a 29 cm; llega
2/10 11:07	Objetivo y amenaza	90	9,9	8,0 s	1 evasión; llega (con video)
2/10 13:46	Independiente, antes	34	10,2	—	Permanece en búsqueda
2/10 13:48	Independiente, antes	60	9,1	—	2 evasiones; no llega
2/10 13:50	Independiente, antes	60	9,1	—	6 evasiones en ciclo
2/10 14:03	Independiente, después	14	10,3	6,9 s	1 evasión; llega
2/10 14:04	Independiente, después	60	10,4	—	4 evasiones; no llega
2/10 14:19	Independiente, después	60	9,5	—	5 evasiones; no llega; imágenes dañadas
2/10 14:20	Independiente, después	45	9,1	5,2 s	Llega; 1 evasión; vuelve a llegar
2/10 14:21	Independiente, después	60	10,0	55,1 s	4 evasiones; llega

A.3 Asignación de pines del ESP32

La Tabla A3 indica las conexiones del microcontrolador. Los pines del sensor ultrasónico y del zumbador se eligieron entre los accesibles en la placa de pruebas; los de sólo entrada quedan reservados para los codificadores.

Tabla A3 Conexiones del ESP32.

Señal	GPIO	Observación
L298N ENA / IN1 / IN2	25 / 26 / 27	Canal A: rueda derecha
L298N ENB / IN3 / IN4	13 / 33 / 32	Canal B: rueda izquierda
Ultrasónico, línea de señal	12	Con divisor 1 k Ω / 2 k Ω
Zumbador	14	Activo, conexión directa
Codificadores (reservados)	34, 35, 36, 39	Sin conectar; sólo entrada